

What Is Convolution?

Convolution is the **fundamental mathematical operation** that describes how a linear time-invariant (LTI) system responds to any input signal.

Core Idea: Every LTI system is completely characterised by its *impulse response* $h[n]$. Given any input $x[n]$, the output $y[n]$ is computed via convolution:

$$y[n] = x[n] * h[n] = \sum_{k=-\infty}^{\infty} x[k] \cdot h[n - k]$$

$$\begin{array}{ccc} \text{Input} & \text{System} & \text{Output} \\ x[n] & \xrightarrow{h[n]} & y[n] = x[n] * h[n] \end{array}$$

Signal Notation

Symbol	Meaning
$x[n]$	Input signal (what you feed into the system)
$h[n]$	Impulse response (characterises the system completely)
$y[n]$	Output signal (system's response to the input)
$\delta[n]$	Unit impulse: 1 at $n = 0$, zero everywhere else
$*$	Convolution operator

Length of the Output

If $x[n]$ has N samples and $h[n]$ has M samples:

$$\text{Output length} = N + M - 1$$

Our Working Example

Given Signals (with Time Origins)

The arrows ($\uparrow$) below each array mark the $n = 0$ **sample**:

$$\begin{array}{ll} x[n] = [2, 3, 4, \underbrace{2, 1, 4, 5}_{\text{negative } n}] & \Rightarrow \text{origin at index 3 (0-based)} \\ & \quad \quad \quad \uparrow \\ h[n] = [\underbrace{1, 2, 3, 4}_{\text{negative } n}] & \Rightarrow \text{origin at index 1 (0-based)} \\ & \quad \quad \quad \uparrow \end{array}$$

Re-Indexed Values

Input Signal $x[n]$		Impulse Response $h[n]$	
n	$x[n]$	n	$h[n]$
-3	2	-1	1
-2	3	0	2
-1	4	1	3
0	2	2	4
1	1		
2	4		
3	5		

Output Time Range

Start of $y[n]$ = start of $x[n]$ + start of $h[n]$ = $(-3) + (-1) = -4$

End of $y[n]$ = end of $x[n]$ + end of $h[n]$ = $(+3) + (+2) = +5$

Length = $7 + 4 - 1 = 10$ samples

The Two Convolution Algorithms

Both algorithms compute **exactly the same output**. They differ only in *perspective*:

	Input Side Algorithm	Output Side Algorithm
Question asked	Where does each input send its energy?	Where does each output gather from?
Direction	Input $\rightarrow$ scatters into output	Output $\leftarrow$ gathers from inputs
Mental model	Broadcasting / Push	Collecting / Pull
Flip needed?	No	Yes — flip $h[n]$
Key formula	$y[n + m] += x[n] \cdot h[m]$	$y[k] = \sum_n x[n] \cdot h[k - n]$

Input Side Algorithm

Concept: Each input sample $x[n]$ creates a *scaled, time-shifted copy* of $h[n]$ in the output. All copies are then added together (superposition).

$$y[n + m] += x[n] \cdot h[m]$$

Physical Intuition

Because $h[n]$ is the system's response to a single unit impulse $\delta[n]$:

- Feeding $x[n_0] = c$ at time n_0 is equivalent to feeding $c\delta[n - n_0]$.
- By linearity: the system outputs $c \cdot h[n - n_0]$.
- The total output is the **superposition** of all such responses.

Decompose $x[n]$ into Impulses

$$x[n] = 2\delta[n + 3] + 3\delta[n + 2] + 4\delta[n + 1] + 2\delta[n] + 1\delta[n - 1] + 4\delta[n - 2] + 5\delta[n - 3]$$

Each Sample Contributes a Shifted Copy of h

$x[n]$	Contribution	$y[-4]$	$y[-3]$	$y[-2]$	$y[-1]$	$y[0]$	$y[1]$	$y[2]$	$y[3]$	$y[4]$	$y[5]$
$x[-3] = 2$	$2 \times [1, 2, 3, 4]$ at -4	2	4	6	8						
$x[-2] = 3$	$3 \times [1, 2, 3, 4]$ at -3		3	6	9	12					
$x[-1] = 4$	$4 \times [1, 2, 3, 4]$ at -2			4	8	12	16				
$x[0] = 2$	$2 \times [1, 2, 3, 4]$ at -1				2	4	6	8			
$x[1] = 1$	$1 \times [1, 2, 3, 4]$ at 0					1	2	3	4		
$x[2] = 4$	$4 \times [1, 2, 3, 4]$ at 1						4	8	12	16	
$x[3] = 5$	$5 \times [1, 2, 3, 4]$ at 2							5	10	15	20
Sum		2	7	16	27	29	28	24	26	31	20

Result: $y[n] = [2, 7, 16, 27, 29, 28, 24, 26, 31, 20]$ for $n = -4$ to $+5$,
 $y[0] = 29$

Step-by-Step Procedure

1. **Identify the start position** of $h[n]$ (here: $n = -1$).
2. **For each sample $x[n]$:**
 - (a) Multiply all values of h by $x[n]$.
 - (b) Place the scaled copy starting at position $n + (\text{start of } h)$.
 - (c) Add the values into the output array.
3. **Sum all contributions** column by column.

Output Side Algorithm

Concept: For each output sample $y[k]$, we look back across all input samples and ask: how much did each one contribute here?

$$y[k] = \sum_n x[n] \cdot h[k - n]$$

This is implemented by **flipping** h and **sliding** it over x .

Physical Intuition

$h[k - n]$ measures how the system at time k responds to an input that arrived at time n :

- $h[0]$: how strongly the system responds *right now*.
- $h[1]$: response strength *one step later*.
- $h[k - n]$: response at time k due to input at time n — a “memory” weight.

The Flip-and-Slide Visualised

1. **Flip** $h[n]$ to obtain $h[-n] = [4, 3, 2, 1]$.
2. **Slide** the flipped filter one position right for each output index k .
3. At each position, compute the **dot product** of overlapping values.

Output Computation Table

k	Calculation	$y[k]$
-4	$x[-3] \cdot h[-1] = 2 \cdot 1$	2
-3	$x[-3] \cdot h[0] + x[-2] \cdot h[-1] = 4 + 3$	7
-2	$x[-3] \cdot h[1] + x[-2] \cdot h[0] + x[-1] \cdot h[-1] = 6 + 6 + 4$	16
-1	$x[-3] \cdot h[2] + x[-2] \cdot h[1] + x[-1] \cdot h[0] + x[0] \cdot h[-1] = 8 + 9 + 8 + 2$	27
0	$x[-2] \cdot h[2] + x[-1] \cdot h[1] + x[0] \cdot h[0] + x[1] \cdot h[-1] = 12 + 12 + 4 + 1$	29
1	$x[-1] \cdot h[2] + x[0] \cdot h[1] + x[1] \cdot h[0] + x[2] \cdot h[-1] = 16 + 6 + 2 + 4$	28
2	$x[0] \cdot h[2] + x[1] \cdot h[1] + x[2] \cdot h[0] + x[3] \cdot h[-1] = 8 + 3 + 8 + 5$	24
3	$x[1] \cdot h[2] + x[2] \cdot h[1] + x[3] \cdot h[0] = 4 + 12 + 10$	26
4	$x[2] \cdot h[2] + x[3] \cdot h[1] = 16 + 15$	31
5	$x[3] \cdot h[2] = 5 \cdot 4$	20

Result: $y[n] = [2, 7, 16, 27, 29, 28, 24, 26, 31, 20]$ for $n = -4$ to $+5$,
 $y[0] = 29$

Time Origin: Why It Matters

The **time origin** (the $n = 0$ reference point) determines *when* the output appears in time. It does **not** change the shape of the output — only its position on the time axis.

Rules for Time-Origin Arithmetic

Property	Formula
Output start index	$n_{\text{start}}^{(y)} = n_{\text{start}}^{(x)} + n_{\text{start}}^{(h)}$
Output end index	$n_{\text{end}}^{(y)} = n_{\text{end}}^{(x)} + n_{\text{end}}^{(h)}$
Position of $y[0]$	Index = $ n_{\text{start}}^{(x)} + n_{\text{start}}^{(h)} $ into the output array

Our Example

$$n_{\text{start}}^{(y)} = (-3) + (-1) = -4, \quad n_{\text{end}}^{(y)} = (+3) + (+2) = +5$$

$y[0]$ is the 5th element (index 4) of the output array $\Rightarrow y[0] = 29$

If the origins had both been at index 0 (i.e. x and h starting at $n = 0$), the output would have been identical in shape but *placed at* $n = 0$ to $n = 9$, with $y[0] = 2$.

Final Verified Answer

n	-4	-3	-2	-1	0	1	2	3	4	5
$y[n]$	2	7	16	27	29	28	24	26	31	20

$$y[n] = [2, 7, 16, 27, 29, 28, 24, 26, 31, 20] \quad n = -4 \text{ to } +5$$

Both algorithms produce the same output. ✓

Key Formulae Reference

Formula	Description
$y[n] = x[n] * h[n] = \sum_{k=-\infty}^{\infty} x[k] h[n-k]$	Definition of discrete convolution
$\text{Length}(y) = \text{Length}(x) + \text{Length}(h) - 1$	Output length
$n_{\text{start}}^{(y)} = n_{\text{start}}^{(x)} + n_{\text{start}}^{(h)}$	Output start time
$x[n] = \sum_k x[k] \delta[n-k]$	Impulse decomposition of any signal
$y[n+m] += x[n] \cdot h[m]$	Input side update rule
$y[k] = \sum_n x[n] \cdot h[k-n]$	Output side formula